A – Parallelepiped

Por ser um paralelepípedo, ele possui 3 grupos de 4 arestas idênticas, cada uma representando comprimento (x), largura (y) e altura (z).

Sabe-se que:

$$a = xy$$
$$b = yz$$
$$c = xz$$

Tirando a raiz quadrada da multiplicação de a, b e c; tem-se:

$$\sqrt{abc} = xyz$$

Para descobrir cada aresta, divide-se o resultado encontrado pelo termo que possui o produto das outras duas arestas, logo:

$$x = \frac{\sqrt{abc}}{b}$$
$$y = \frac{\sqrt{abc}}{c}$$
$$z = \frac{\sqrt{abc}}{a}$$

Tendo esses valores, é possível concluir que:

$$S = 4 * \sqrt{abc} * \left(\frac{1}{a} + \frac{1}{b} + \frac{1}{c}\right)$$

Complexidade esperada: O(1)

M – A-Sequence

Para garantir que a sequência é uma A-Sequence é preciso garantir duas condições:

- $1 \leq a_1 < a_2 < \cdots < a_n$
- a_i *não pode ser a soma de dois ou mais termos anteriores da sequência*

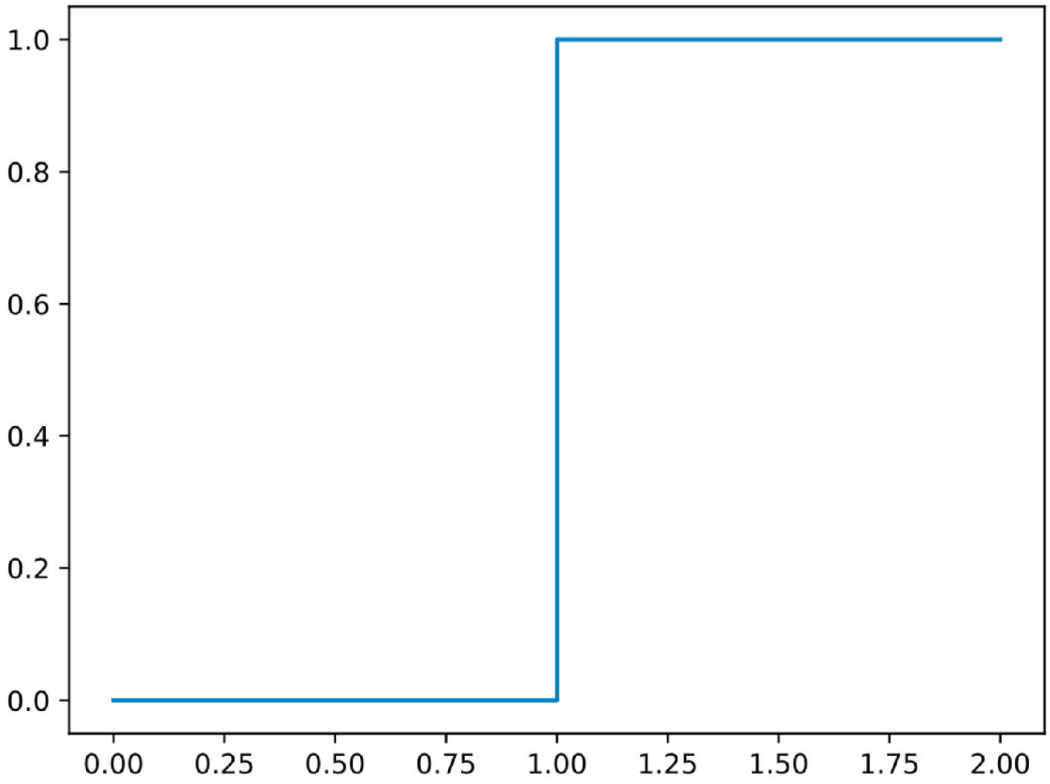
Para garantir a primeira condição, é só garantir que a sequência é crescente (a entrada já garante que os números vão de 1 a 1000, então não há a necessidade de checar se o primeiro elemento é maior ou igual a 1)

Para garantir a segunda condição, é preciso saber quais as somas dos subconjuntos da sequência de 1 a i-1 são possíveis. É possível obter essas somas utilizando o algoritmo Subset Sum, verificando se a soma é possível e caso seja, não é uma A-Sequence, caso contrário, atualizar o vetor do Subset Sum com o elemento atual e testar para o próximo elemento. Caso nenhuma soma seja possível, a sequência é uma A-Sequence.

Complexidade esperada: O(NS), onde S é a soma máxima de todos os elementos

C – WiFi

Imagine que você tem um valor de distância D (não necessariamente mínimo) que seja suficiente para prover sinal a todos os residentes usando até n pontos de acesso. Sabe-se que se for utilizado um valor D maior, ele também será capaz de prover sinal a todas as casas usando até n pontos de acesso (pois se colocarmos os pontos de acesso nas mesmas posições anteriores, aumentará o alcance total). Supondo que você esteja no caso de distância mínima, todas as distâncias maiores que essa será possível prover o sinal aos residentes e todas as distâncias menores não será possível, imaginando isso como uma função binária, o resultado seria o apresentado no gráfico abaixo:



Logo, a função é monotônica não-decrescente e por isso é possível aplicar a busca-binária em f(D).

Para que f(D) possa representar com exatidão que é possível prover sinal com aquela distância, deve-se colocar o primeiro access points a uma distância D a direita do primeira residência descoberta de sinal, esse access point cobrirá todas as casas que estão a distância de até 2D do primeiro access point descoberto, após isso repete-se o processo e se for possível cobrir todas as casas com até n access points, a função retorna 1, caso contrário retorna 0.

Complexidade esperada: O(Nlog(N)log(Dmax/RES)), onde Dmax é 1000000 e RES é a resolução: 0.1

Obs: O fator Nlog(N) é bastante amortecido porque se a função f(D) for feita de maneira inteligente, os lower_bounds utilizados para descobrir o próximo elemento vão ser feitos em um intervalo menor.

Obs2: O vetor precisa ser ordenado previamente

D – Tapioca And The Chocolate Rain

Pode-se modelar o problema como um grafo onde cada vértice é um tripla (x, y, t) e está conectado aos vértices (x-1, y, t+1), (x+1, y, t+1), (x, y-1, t+1), (x, y+1, t+1) e (x, y, t+1), realizando uma busca em profundidade, verifica-se o caminho onde Tapioca pode comer o maior número de trufas.

Complexidade esperada: $\boldsymbol{O(x_{max}y_{max}t_{max})}$

E – A Terra Herdada por Tio Tom

Observe que o grafo das células que estão vazias pode ser representado por um grafo bipartido, onde se um ponto está no conjunto U, todos os pontos aos quais ele está conectado estão em um conjunto V e se um ponto está no conjunto V, todos os pontos aos quais ele está ligado estão no conjunto U. O objetivo desse problema é descobrir o acoplamento máximo no grafo bipartido, para resolver este problema com um número de vértices que pode chegar até a 10^4, utiliza-se o algoritmo de Hopcroft-Karp.

Complexidade esperada: $\boldsymbol{O(\sqrt{V}E)}$

L – D-Query

Existem várias maneiras de resolver esse problema, a abordagem utilizada será utilizando o algoritmo de Mo. O algoritmo de Mo basicamente distribui as queries em blocos de $\sqrt{q}$ queries, onde uma query está em um bloco se $\frac{L}{\sqrt{q}}$ for igual ao índice do bloco, dentro desses blocos, as queries estão ordenadas em ordem crescente no valor de R. Ou seja, as queries serão ordenadas primeiramente em ordem crescente pelo índice do bloco e, se houver empate, em ordem crescente pelo valor de R.

As queries então são processadas em uma ordem diferente e são calculadas utilizando dois ponteiros, um que representa o ponteiro l que representa o lado esquerdo do intervalo e o ponteiro r que representa o lado direito do intervalo, a ideia é que esse intervalo adapte seu tamanho para o intervalo da query (com o auxílio de um vetor de frequências).

Exemplo:

O ponteiro l está na posição 4 e o ponteiro r está na posição 7, o intervalo da minha query é [3, 9], logo o ponteiro l deve ir para a posição 3 (adicionando então o valor do elemento da posição 3 ao vetor de frequência) e o ponteiro r deve ir para a posição 9 (adicionando então o valor do elemento das posições 8 e 9 ao vetor de frequência), toda vez que um valor do vetor de frequência muda de 0 para 1, a resposta do intervalo é incrementada e toda vez que um valor do vetor de frequência muda de 1 para 0, a resposta do intervalo é decrementada; quando o intervalo atual é igual ao intervalo da query é a resposta do intervalo é a resposta da query (as respostas das queries devem ser apresentadas na ordem original, então ao ordenar as queries, é preciso manter o índice original delas)

Complexidade esperada: $\boldsymbol{O(q\sqrt{q})}$

K – Goemon em Apuros

A diferença entre cada ponto e a origem da explosão de luz pode ser representada por um vetor e desse vetor, a informação mais importante para esse problema é o ângulo, observe que se o ângulo de Goemon está entre o ângulo do ponto inicial da parede mais próximo e o ângulo do ponto final da parede mais próxima, ele está fora do campo de visão e portanto está seguro. Para cada ponto que Goemon pode estar, deve ser feita uma busca binária para saber o maior ponto inicial de uma parada que tenha ângulo menor ou igual ao ângulo de Goemon e incrementar a resposta caso o ângulo de Goemon seja menor ou igual ao ângulo do ponto final da parede. É possível achar a parede com essa característica em log(P) utilizando busca binária.

Complexidade esperada: $O(Glog(P))$

Obs: O problema não diz o limite de P, mas independente do valor (depende apenas para ter que alocar o número de posições do vetor de paredes), não estouraria o time limit pois o fator G é menor ou igual a 10^4

Obs2: Considerei $P \leq 10^4$ e passou. XD

H – Digitando no Telefone Celular

Cria-se uma trie com as palavras do caso de teste e comprime-se a trie com um grafo auxiliar de forma que se há uma sequência de nós tal que todos os nós tenham grau de saída menor ou igual a 1, eles viram apenas um nó. Observando esse grafo auxiliar, é fácil perceber que a resposta é igual à soma de todos os valores dos nós dividido pelo número de palavras, onde o valor de um nó é igual ao número de folhas na sua sub-árvore (nesse caso ou se exclui o valor da raíz ou o valor das folhas, pois essa contagem é feita repetida, ou simplesmente só subtrai N, dá no mesmo).

Complexidade esperada: $O(S)$ onde S é a soma do comprimento de todas as palavras

G – Catálogo de Músicas

Observe que a estrutura apresentada pode ser representada por uma árvore onde cada nó folha é um arquivo e qualquer outro nó é uma pasta. O problema dá uma restrição importante onde o número de pastas na entrada é menor ou igual a 10^5, como o número de arquivos também é menor que 10^5, tem-se que o número máximo de nós é $2 * 10^5 + 1$ (contando a raiz). Cada string deve ser processada e separada em pastas e arquivos (caso as pastas não existam ainda, deve-se criar os seus nós, é possível armazenar os nomes das pastas em um mapa para saber se nó já foi criado ou não) e para facilitar a resolução deve-se considerar que a / faz parte do nome da pasta.

Faz-se então uma busca na árvore, para cada nó haverá um valor associado ao número de caracteres utilizados para apresentar o conteúdo da árvore (observe que na raiz, esse valor é a soma dos tamanhos das palavras), esse número de caracteres do nó é equivalente ao número de caracteres do pai dele menos o tamanho no nome da pasta vezes o número de folhas nessa subárvore mais 3 vezes o número de folhas fora dessa subárvore (o número 3 é dado pelo número de ../ que serão adicionados), o menor dentre todos esses números de caracteres é a resposta.

Complexidade esperada: $O(N + M)$, onde N é o número de arquivos e M é o número de pastas

I – Mantenha Isso Energizado

Esse problema é mais fácil de ser analisado do final, considera-se que existe um nível N+1 que caso se chegue a ele, foi possível passar por todos os níveis.

Ordena-se as lojas em ordem decrescente pelo nível e cria-se um vetor que para todos os valores menores que N+1 assuma um valor INF (uma constante muito alta, maior que qualquer resposta possível) e para N+1 o valor 0, esse vetor representa o custo mínimo de ir desse nível para o nível N+1 e completar todos os níveis. Para cada loja, verifica-se o nível máximo que o jogador pode ir usando apenas a energia dessa loja, essa loja será chamada de R, verifica-se então no intervalo [L+1, R], o custo mínimo para completar o jogo que será chamado de X, o custo mínimo do nível L será o mínimo entre o custo mínimo atual do nível L e o custo da energia da loja em questão mais X. Para realizar as pesquisas nos intervalos pode ser utilizada uma estrutura de Range Minimum Query que tenha uma complexidade de pesquisa e atualização de até log(n) como a Segment Tree.

Complexidade esperada: $O(Llog(n))$

F – O Último Dígito Não-Zero

Esse problema basicamente requer o último dígito significativo do arranjo de N com M posições que pode ser representado por $\frac{N!}{(N-M)!}$. Para resolver esse problema utilizando a abordagem mais simples, é necessário pré-processar todos os fatoriais e armazenar suas informações em três vetores o primeiro vetor (P) representa o resto do produto acumulado de todos os elementos i de 1 até x retirando os fatores 2 e 5 por 10, o segundo vetor (V2) representa o expoente de 2 contido no fatorial de x e o terceiro vetor (V5) representa o expoente de 5 contido no fatorial de x. Para facilitar o entendimento foi construída uma tabela com os valores do vetor de 1 até 10.

	1	2	3	4	5	6	7	8	9	10
P	1	1	3	3	3	9	3	3	7	7
V2	0	1	1	3	3	4	4	7	7	8
V5	0	0	0	0	1	1	1	1	1	2

É importante observar que o valor dos elementos de P é sempre ímpar e diferente de 5, o que dá um característica muito importante a eles, eles sempre irão possuir um inverso multiplicativo com módulo 10, pois sempre serão números coprimos, então $\frac{N!}{(N-M)!}\%10$ equivale a:

$$X = \frac{P[N] * 2^{V2[N]} * 5^{V5[N]}}{P[N-M] * 2^{V2[N-M]} * 5^{V5[N-M]}}\%10$$

$$X = P[N] * (P[N-M])^{-1} * 2^{V2[N]-V2[N-M]} * 5^{V5[N]-V5[N-M]} \,\%\, 10$$

Como não se está interessado diretamente em X (pois muito provavelmente ele será 0), remove-se os fatores que geram os zeros (ou seja as potências de 10).

Se $V2[N] - V2[N-M] > V5[N] - V5[N-M]$:

$$Y = P[N] * (P[N-M])^{-1} * 2^{V2[N]-V2[N-M]-(V5[N]-V5[N-M])} \,\%\, 10$$

Caso contrário:

$$Y = P[N] * (P[N-M])^{-1} * 5^{V5[N]-V5[N-M]-(V2[N]-V2[N-M])} \,\%\, 10$$

Onde Y é a resposta desejada.

Complexidade esperada: O(N+Q), onde Q é o número de queries

Obs: O valor da tripla (P[0], V2[0], V5[0]) deve ser (1,0,0)

Obs2: Os inversos multiplicativos de 1, 3, 7 e 9 podem ser pré-calculados (inclusive não precisa pensar muito para descobrir quem eles são)

Obs3: Existe uma solução com complexidade O(Qlog(N)) que será discutida no final.

J – Onze

Para resolver esse problema será explorado o critério de divisibilidade por 11, onde se um número é divisível por 11, a soma dos dígitos das posições ímpares (unidades, centenas, ...) é igual à soma dos dígitos das posições pares(dezenas, milhares, ...) módulo 11, no caso dessa questão, é melhor organizar a equação de tal forma que a soma dos dígitos das posições ímpares menos a soma dos dígitos das posições pares seja igual a 0.

Pode se então observar o número como a composição de dois conjuntos, um conjunto U das posições ímpares com tamanho $\left\lceil \frac{dig(N)}{2} \right\rceil$ e outro conjunto V as posições pares com tamanho $\left\lfloor \frac{dig(N)}{2} \right\rfloor$, os quais será feita a separação dos dígitos do número (dig(N) é a função que retorna o número de dígitos de N).

Observe que se a separação dos dígitos de N for válida, a solução para esses conjuntos será igual ao número de todas as reordenações **diferentes** do conjunto U multiplicado pelo número de todas as reordenações **diferentes** do conjunto V. Ou seja:

$$g(U) = \frac{\left\lceil \frac{N}{2} \right\rceil !}{d_{0i}!\, d_{1i}!\, d_{2i}!\, d_{3i}!\, d_{4i}!\, d_{5i}!\, d_{6i}!\, d_{7i}!\, d_{8i}!\, d_{9i}!}$$

$$g(V) = \frac{\left\lfloor \frac{N}{2} \right\rfloor !}{d_{0p}!\, d_{1p}!\, d_{2p}!\, d_{3p}!\, d_{4p}!\, d_{5p}!\, d_{6p}!\, d_{7p}!\, d_{8p}!\, d_{9p}!}$$

$$f(U,V) = g(U) * g(V)$$

Onde d_{ki} significa o número de dígitos k contidos no conjunto dos números ímpares e d_{kp} significa o número de dígitos k contidos no conjunto dos números pares, essa fórmula é a de permutação com elementos repetidos, logo são válidas as seguintes equações:

$$d_{0i} + d_{1i} + d_{2i} + d_{3i} + d_{4i} + d_{5i} + d_{6i} + d_{7i} + d_{8i} + d_{9i} = \left\lceil \frac{N}{2} \right\rceil$$

$$d_{0p} + d_{1p} + d_{2p} + d_{3p} + d_{4p} + d_{5p} + d_{6p} + d_{7p} + d_{8p} + d_{9p} = \left\lfloor \frac{N}{2} \right\rfloor$$

Ou seja deseja-se encontrar a soma de todos os f(U, V) tais que a soma dos dígitos de U menos a soma dos dígitos de V módulo 11 seja 0.

Para facilitar a análise abaixo, será utilizado o valor M para representar o valor de $\left\lfloor \frac{N}{2} \right\rfloor$ ou $\left\lceil \frac{N}{2} \right\rceil$ e que será omitida a letra i ou p (a fórmula abaixo serve para outras casos.

Observe que as fórmulas apresentadas são equivalentes a:

$$g(U) = \prod_{i=0}^{9} \binom{M - \sum_{j=0}^{i-1} d_j}{d_i}$$

Tendo essas informações em mente, é possível montar uma matriz pd[10][11][101][101] onde o primeiro estado representa o dígito que será colocado nos conjuntos, o segundo estado representa o resto da diferença entre as somas dos dígitos dos conjuntos módulo 11, o terceiro estado representa o número de dígitos que faltam ser colocados no conjunto U e o quarto estado representa o número de dígitos que faltam ser colocados no conjunto V.

O algoritmo da recursão seria o seguinte:

$solve(posicao, resto, NU, NV)$
  $ans \leftarrow 0$
  $se\ pos = 0\ entao$
    $se\ res = 0\ retorne\ 1$
    $senao\ retorne\ 0$
    $fimse$
  $fimse$
  $para\ i\ de\ 0\ ate\ dig[pos]\ faca$
    $se\ (NU - i) \geq 0\ e\ (NV - (dig[pos] - i) \geq 0\ entao$

$$ans \leftarrow ans + \binom{NU}{i} * \binom{NV}{dig[pos] - i} * solve\Big(pos + 1, \Big(resto + pos * \big(i - (dig[pos] - i)\big)\Big)\%11, NU - i, NV - (dig[pos] - i)\Big)$$

    $fimse$
  $fimpara$

Complexidade esperada: $O(B * MOD * dig(N)^2)$, onde B é a base 10, MOD é o resto 11

Curiosidade: Quando eu tentei fazer, fiquei bastante tempo travado tentando formular a equação que relaciona a permutação dos elementos de um conjunto como um produtório de combinações, acabou sendo um resultado bastante satisfatório para PDs que utilizem combinações de subconjuntos válidas. :D

F – O Último Dígito Não-Zero : $O(Qlog(N))$

Para resolver essa questão em Qlog(N) é primeiro preciso saber calcular o último dígito não-zero de um fatorial, existe alguns resultados conhecidos na internet como o que pode ser visto nesse link:

https://www.campusgate.co.in/2013/10/finding-last-two-non-zero-digits-of.html

No entanto, será discutido aqui a solução log(N) que eu deduzi quando resolvi esse problema.

Observe que a sequência de 1 a N pode ser decomposta em sequências de números cujo expoente de 5 é menor ou igual a k, onde $k \geq 0$, o valor de k define basicamente a sequência.

Exemplo:

Sequência original [1,30]

Sequência 0: {1, 2, 3, 4, 6, 7, 8, 9, 11, 12, 13, 14, 16, 17, 18, 19, 21, 22, 23, 24, 26, 27, 28, 29}

Sequência 1: {5, 10, 15, 20, 30}

Sequência 2: {25}

Observe também que se cada sequência k for dividida por 5^k, as sequências terão o mesmo comportamento da sequência 0.

Agora, a otimização por trás dessa divisão em sequências é que:

$$1 * 2 * 3 * 4 * 6 * 7 * 8 * 9 (mod 10) \equiv 6$$
$$6^r (mod 10) \equiv 6$$

Ou seja, pode-se separar mais ainda cada sequência em blocos de 10 elementos consecutivos e o produto dessas subsequências será 6, transformando isso em uma função recursiva ficaria:

$func(N)$
  $se\ N = 0$
    $retorne\ (1, 0)$
  $fimse$
  $x = func\left(\frac{N}{5}\right)$
  $se \frac{N}{10} > 0\ entao$
    $retorne\ \left(6 * prod[N\%10] * x[0]\ \%\ 10, \frac{N}{5} + x[1]\right)$
  $senao$
    $retorne\ \left(prod[N\%10] * x[0]\ \%\ 10, \frac{N}{5} + x[1]\right)$
  $fimse$

O vetor prod é o produto dos elementos da sequência 1, 2, 3, 4, 6, 7, 8, 9.

	0	1	2	3	4	5	6	7	8	9
prod	1	1	2	6	4	4	4	8	4	6

Por conveniência, admite-se que o valor de prod[0] é 1 e o valor de prod[5] é igual ao de prod[4], pois esses elementos não estão na sequência.

O valor de func(N, 1) retorna uma tupla onde o segundo elemento K é o expoente de 5 de N!, e o primeiro elemento X é o último dígito não-zero de $\frac{N!}{5^k}$

Como se sabe que todo fatorial tem o expoente do fator 2 maior que o expoente do fator 5, sabe-se que todos os últimos dígitos não-zero do fatorial são pares para N > 1, para calcular o último dígito não-zero de N!, pode-se simplesmente dividir por 2^k. Não há inverso multiplicativo para 2 com módulo 10, mas empiricamente sabe-se que só podem ser dois resultados distintos.

$2 \rightarrow 1\ e\ 6$
$4 \rightarrow 2\ e\ 7$
$6 \rightarrow 3\ e\ 8$
$8 \rightarrow 4\ e\ 9$

Só que como o último dígito de N! é par, pode-se interpretar a divisão como uma volta no ciclo $2 \rightarrow 4 \rightarrow 8 \rightarrow 6 \rightarrow 2$ e aí só teria que voltar o valor de X K vezes no ciclo para obter o valor desejado (observe que se X for 1, ele não vai entrar no ciclo pois K é zero).

Resolvido então o último-dígito não-zero do fatorial, prossegue-se para o último dígito não-zero do arranjo. Para isso, acha-se o número de dígitos não zero de N e N-M, que serão chamados de X1 e X2, além disso deve-se achar o expoente de 2 relacionado a N e o expoente de 2 relacionado a N-M, além dos expoentes de 5 relacionados a N e N-M (que são encontrados juntos a X1 e X2).

Para achar os expoentes de 2, pode-se usar uma função muito semelhante a func passando os mesmos parâmetros, essa função pode ser vista no procedimento abaixo:

$func2(N)$
  $se\ N = 0$
    $retorne\ 0$
  $fimse$
  $retorne\ \frac{N}{2} + func2\left(\frac{N}{2}\right)$

Agora pode-se aproveitar o resultado que foi encontrado na resolução normal:

Se $V2[N] - V2[N-M] > V5[N] - V5[N-M]$:

$$Y = P[N] * (P[N-M])^{-1} * 2^{V2[N]-V2[N-M]-(V5[N]-V5[N-M])} \% 10$$

Caso contrário:

$$Y = P[N] * (P[N-M])^{-1} * 5^{V5[N]-V5[N-M]-(V2[N]-V2[N-M])} \% 10$$

O mais interessante disso é que sabendo o último dígito não-zero de N!, é possível recuperar o valor de P[N] !!!!!! :D

Pois:

$$X = P[N] * 2^{V2[N]-V5[N]} \% 10$$

Do mesmo jeito que se usou o ciclo pros números pares antes, usa-se ele novamente para voltar o valor de X $V2[N] - V5[N]$ vezes no ciclo, só que como se sabe que P[N] é ímpar, esse resultado será acrescido de 5 (Por quê? Descubra).

Com os valores de P[N], V2[N], V5[N], P[N-M], V2[N-M] e V5[N-M] recuperados em log(N) é possível calcular Q queries em tempo **Qlog(N)**

UAUUUUUU!!!